

LAPORAN PRAKTIKUM
SISTEM OPERASI

MODUL XIV
KEAMANAN



Universitas
Telkom

Disusun Oleh:

GHAZA ZIDANE NURRAIHAN

2311104038

SE-07-01

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2025

Tujuan:

1. Mahasiswa mampu memahami Sistem keamanan di Linux.

Catatan:

1. **Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir.**
2. **Untuk soal source code, kumpulkan SS-nya saja.**
3. **Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username.**
4. **Berikan identitas nama - nim dalam bentuk comment di Source Code.**
5. **Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!**

Jurnal

1. Integritas: dasar hashing [10 Point]

- a. **[2 Point]** Lakukan hash SHA256, SHA512 dan MD5 untuk file /etc/passwd. Berapa nilai hash dari file /etc/passwd? Screenshot nilai hash dari file tersebut.

```
sha256sum /etc/passwd
sha512sum /etc/passwd
md5sum /etc/passwd
```

```
<nilai_hash_sha256> /etc/passwd
<nilai_hash_sha512> /etc/passwd
<nilai_hash_md5> /etc/passwd
```

- b. **[2 Point]** Buatlah file bernama test_0.txt pada folder /home/praktikan. Isi file tersebut isi yang ada di file /etc/passwd (copy paste **isi** file /etc/passwd ke test_0.txt)

```
cp /etc/passwd test_0.txt
ls
```

- c. **[2 Point]** Lakukan hash SHA256, SHA512 dan MD5 untuk file test_0.txt. Berapa nilai hash dari file test_0.txt? Screenshot nilai hash dari file test_0.txt.

```
sha256sum test_0.txt
sha512sum test_0.txt
md5sum test_0.txt
```

- d. **[2 Point]** Rename file test_0.txt menjadi file_0.txt. Lakukan hash SHA256, SHA512 dan MD5 untuk file_0.txt. Berapa nilai hash file_0.txt? Screenshot nilai hash dari file_0.txt

```
mv test_0.txt file_0.txt
sha256sum file_0.txt
sha512sum file_0.txt
md5sum file_0.txt
```

- e. **[2 Point]** Apa hasil pengamatan Anda? File apa saja yang mempunyai hash yang sama?

Jelaskan

jawab:

File /etc/passwd, test_0.txt, dan file_0.txt memiliki nilai hash yang sama karena isi file tidak berubah. Algoritma hash menghitung berdasarkan isi data, bukan berdasarkan nama file. Jadi, meskipun file disalin atau diganti namanya, selama isi file tetap sama, nilai hash juga tetap sama.

2. Integritas: avalanche [15 Point]

- a. **[3 Point]** Download file bernama test_1.txt di link ini tiny.cc/test1_txt

```
wget tiny.cc/test1_txt -O test_1.txt
```

- b. **[3 Point]** Lakukan hash SHA256, SHA512 dan MD5 untuk file test_1.txt. Berapa nilai hash test_1.txt? Screenshot nilai hash dari file test_1.txt.

```
sha256sum test_1.txt
sha512sum test_1.txt
md5sum test_1.txt
```

- c. **[3 Point]** Hapuslah titik diakhir file test_1.txt tersebut, simpan file tersebut!

```
nano test_1.txt
```

- d. **[3 Point]** Lakukan hash dari SHA256, SHA512 dan MD5. Screenshot nilai hash dari file test_1.txt.

```
sha256sum test_1.txt
sha512sum test_1.txt
md5sum test_1.txt
```

- e. **[3 Point]** Apa analisis (hasil pengamatan) Anda mengenai hal tersebut! Apakah nilai hash sama?

jawab

Nilai hash berubah total setelah satu karakter saja dihapus. Hal ini menunjukkan Avalanche Effect, yaitu perubahan kecil pada input akan menghasilkan output hash yang sangat berbeda. Oleh karena itu, hash dapat digunakan untuk mendeteksi perubahan kecil pada file.

3. Integritas: metadata [18 Point]

- a. **[3 Point]** Download file bernama test_1.doc di link ini tiny.cc/test1_doc

```
wget tiny.cc/test1_doc -O test_1.doc
```

- b. **[3 Point]** Lakukan hash SHA256, SHA512 dan MD5 untuk file test_1.doc. Screenshot nilai hash dari file test_1.doc.

```
sha256sum test_1.doc
sha512sum test_1.doc
md5sum test_1.doc
```

- c. Buka kembali file test_1.doc. Lakukan hal ini: **[3 Point]** Ketik abcdef. Save test_1.doc ii. **[3 Point]** Hapus abcdef. Save test_1.doc

```
nano test_1.doc
```

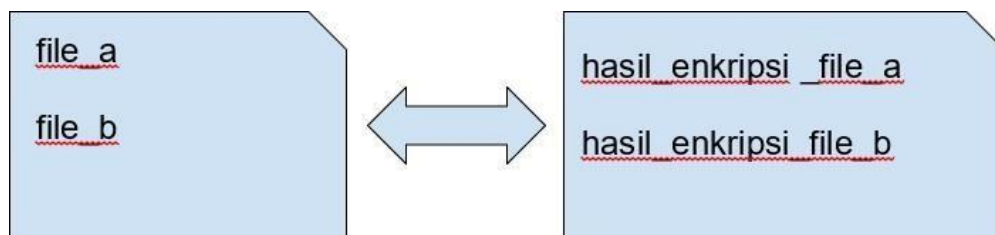
- d. **[3 Point]** Lakukan hash SHA256, SHA512 dan MD5 untuk file test_1.doc. Screenshot nilai hash dari file test_1.doc.

```
sha256sum test_1.doc
sha512sum test_1.doc
md5sum test_1.doc
```

- e. **[3 Point]** Hasil pengamatan apa yang diperoleh? Jelaskan alasannya
Nilai hash berubah meskipun isi dokumen dikembalikan seperti semula. Hal ini terjadi karena file dokumen menyimpan metadata, seperti waktu modifikasi, informasi penyimpanan, atau struktur internal file. Ketika file disimpan ulang, metadata ikut berubah sehingga nilai hash file juga berubah.

4. Konfidensialitas: encfs [27 Point]

Encfs adalah tool untuk melakukan mount dan membuat file system yang terenkripsi. Cara kerjanya adalah sebagai berikut. Ada 2 folder yang akan dibuat, dua folder tersebut saling terkait. Satu folder merupakan folder yang tidak terenkripsi, folder lainnya merupakan hasil enkripsi dari folder pertama.



WARNING: INSTALL ENCFs!

```
sudo apt-get update
sudo apt-get -y install encfs
```

Setelah terinstall silakan melanjutkan pengerjaan jurnal.

- a. **[5 Point]** Jalankan perintah berikut:

```
encfs ~/folder_anda/folder_terenkripsi ~/folder_anda/folder_normal
```

Pilih y (enter), pilih y (enter), enter, kemudian buatlah password. **Ingatlah password yang dibuat.** Setelah selesai, perhatikan bahwa telah muncul folder bernama folder_normal dan folder_terenkripsi.

```
mkdir folder_terenkripsi folder_normal
encfs ~/ghaza_2311104038/folder_terenkripsi ~/ghaza_2311104038/folder_normal
```

- b. [5 Point] Copy dua atau tiga buah file (file apa saja) ke folder_normal. Amati dan tulis hasil observasi Anda pada folder_terenkripsi!

```
cp /etc/passwd folder_normal/

echo "File rahasia Ghaza" > folder_normal/rahasia.txt

ls folder_normal

ls folder_terenkripsi
```

File yang dimasukkan ke folder_normal dapat terlihat secara normal di folder tersebut. Namun, pada folder_terenkripsi, file tampil dengan nama acak atau bentuk terenkripsi. Hal ini menunjukkan bahwa EncFS menyimpan data asli dalam bentuk terenkripsi.

- c. 5 Point] Hapus salah satu file (bebas) pada folder_terenkripsi. Amati dan tulis hasil observasi Anda pada folder_normal!

```
ls folder_terenkripsi
rm folder_terenkripsi/*
ls folder_normal
```

- d. [5 Point] Lakukan umount dengan perintah:

```
Fusermount -u ~/folder_anda/folder_normal
```

Amati dan tulis hasil observasi Anda pada folder_normal dan folder_terenkripsi

```
fusermount -u ~/ghaza_2311104038/folder_normal

ls folder_normal

ls folder_terenkripsi
```

- e. [7 Point] Buatlah folder baru bernama folder_sembarang. Lakukan perintah berikut ini:

```
encfs ~/folder_anda/folder_terenkripsi ~/folder_anda/folder_sembarang
```

Amati dan tulis hasil observasi Anda!

```
mkdir folder_sembarang

encfs ~/ghaza_2311104038/folder_terenkripsi ~/ghaza_2311104038/folder_sembarang

ls folder_sembarang
```

File yang sebelumnya tersimpan dalam bentuk terenkripsi dapat kembali diakses melalui folder_sembarang setelah dimount menggunakan password yang benar. Hal ini menunjukkan bahwa data terenkripsi dapat dibaca kembali selama kunci atau password yang digunakan sesuai.

5. Konfidensialitas: gpg [30 Point]

- a. [5 Point] **Membuat kunci publik dan privat.** Jalankan perintah ini:

```
gpg --gen-key
```

Baca dan ikuti perintah yang ada dari program gpg!

```
gpg --gen-key
```

```
Real name: Ghaza Zidane Nurraihan  
Email address: ghaza@example.com
```

- b. [5 Point] Hasil publickey dan privatekey ada di folder ~/.gnupg. Privatekey tidak boleh keluar dari komputer ini dan hanya Anda saja yang dapat mengaksesnya. Kunci publik akan dibagikan kepada seluruh dunia atau untuk orang yang Anda inginkan saja. Kunci publik adalah pubring.gpg dan kunci private adalah secring.gpg

Lakukan perintah berikut ini untuk mengetahui daftar key yang Anda punya dan fingerprint

kunci yang Anda punya!

```
gpg --list-keys gpg --fingerprint nama@email.com
```

Screenshot hasil perintah di atas!

```
gpg --list-keys  
gpg --fingerprint ghaza@example.com
```

- c. [5 Point] **Export kunci publik Anda.** Jalankan perintah ini:

```
gpg --armor --export nama@email.anda > mypublic_key.asc
```

File **mypublic_key.asc** adalah file yang akan dibagikan kepada teman-teman Anda. Teman Anda akan menggunakan kunci publik Anda jika ingin mengirim pesan rahasia kepada Anda. Hanya Anda yang dapat membaca pesan tersebut karena hanya

mempunyai kunci privat yang bersesuaian dengan kunci publik Anda.

- i. Rename mypublic_key.asc menjadi nim_anda.asc, Contoh: 130118xxxx.asc ii.

Taruh file nim_anda.asc ke folder pada link berikut: <http://tiny.cc/SisopNomor5C>

```
gpg --armor --export ghaza@example.com > mypublic_key.asc
mv mypublic_key.asc 2311104038.asc
ls
```

- d. **[5 Point] Mengimport kunci publik orang lain.** Silakan download file `nim_teman_sebelah_anda.asc` dan jalankan perintah ini untuk mengimport (menambahkan kunci publik orang lain ke sistem Anda):

```
gpg --import nim_teman_sebelah_anda.asc
```

Contoh: “`gpg --import 130118yyyy.asc`” Setelah mengimport kunci publik teman Anda, Anda dapat mengirimkan pesan kepada teman Anda secara terenkripsi. Untuk memastikan bahwa kunci publik telah diimport lakukan perintah

```
gpg --list-keys
```

dan nama teman Anda ada pada hasil perintah tersebut.

```
gpg --import 2311104038.asc
gpg --list-keys
```

- e. **[5 Point] Enkripsi pesan.** Buatlah file bernama `file_rahasia.txt`. Isi file tersebut dengan pesan rahasia Anda. Pesan inilah yang akan Anda kirim ke teman Anda. Jalankan perintah ini untuk melakukan enkripsi:

```
//ditulis dalam satu baris gpg --encrypt --armor -r
alamat_email_teman_anda_yang_baru_diimport@xxx.com
```

```
nano file_rahasia.txt
```

- a. **[5 Point]** Hanya teman anda yang dapat membuka file tersebut. Hasil dari proses tersebut adalah `file_rahasia_untuk_teman_anda.asc` (contoh: `file_rahasia_untuk_130118yyyy.asc`) Taruh `file_rahasia_untuk_teman_anda.asc` ke folder pada link: <http://tiny.cc/SisopNomor5F>

Download dan buka file yang diperuntukkan bagi Anda dan lakukan dekripsi untuk melihat isi pesan dengan perintah:

```
gpg file_rahasia_nim_anda.asc
```

```
gpg ghaza_rahasia_untuk_2311104028.asc
cat ghaza_rahasia_untuk_2311104038
```

REFERENSI

<https://en.wikipedia.org/wiki/Xinu>